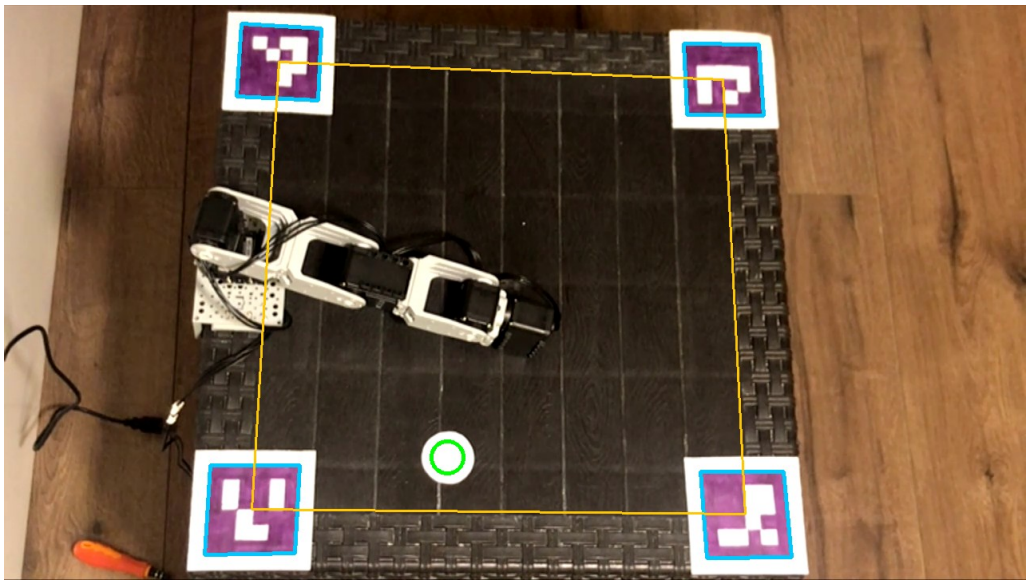
Vision-Based Dot Touching with a Leader–Follower Robot Arm

Data collection, modelling and deployment on the real arm

Technical Report



ArUco markers at the table corners, the table quadrilateral built with a homography, and the detected white dot.

Dynamixel AX-12A · OpenCM9.04 · OpenCV · Python

October 2026

Abstract

In this work a second, **follower** arm was built that copies every motion of a hand-guided **leader** robot arm (five Dynamixel AX-12A servos each, OpenCM9.04 controller). Using the leader, the motion of touching a white dot on the table was recorded 119 times; each recording stores the joint positions of the leader and the follower at about 100 Hz and the position of the dot on the table, measured by a camera looking down from above. The dot position was converted into a table coordinate *(u, v)* in the range 0–1 using four ArUco markers placed at the table corners and a **homography**.

From the 114 episodes left after cleaning, a regression model was trained that maps the dot position to the touch pose. Leave-one-out (LOO) validation selected degree-3 polynomial ridge regression. The base joint was learned with an error of about 1.4° (4.6 units). Although the individual errors of the shoulder, elbow and wrist joints look large, 95.4% of that error was shown to lie in a single direction, which comes from the **redundancy** of the three-link arm (the same point can be reached with several poses); the actual error that moves the end point was found to be about 3.1°, which corresponds to at most about 1.8 cm for a 34 cm arm.

The predicted poses were executed on the real follower arm with **minimum-jerk** trajectories (approach, slow descent, lift, return). 9 runs were completed on the real arm; according to the user's observation the arm found the dot every time. However, the results were not measured in millimetres; this and other open points are listed in the limitations section of this report.

Table 1. *Key figures*

Quantity	Value
Recorded / used episodes	119 / 114 (5 episodes dropped because they were recorded on a nearly empty battery)
Number of samples used	172,814 (≈ 100 Hz)
Episode duration	median 15.0 s (8.8–26.8 s)
Follower–goal difference	median 2 units, 95% ≤ 6 units (≈ 1.8°)
LOO error: base	4.6 units ≈ 1.4°
LOO error: outside the redundancy direction (shoulder–elbow–wrist)	10.6 units ≈ 3.1°
Estimated end-point error (calculated)	≈ 1–1.8 cm (not measured)
Runs completed on the real arm	9 (17 records in total; 8 runs aborted before moving)

Contents

Abstract 2

1. Introduction 5

1.1 Goal and scope 5

1.2 Overall approach 5

2. System architecture 6

2.1 Hardware 6

2.2 Software components and data flow 6

3. Servo basics 8

3.1 Units 8

3.2 Dead zone 8

3.3 Power and temperature 8

4. Leader–follower control 9

4.1 Joint mapping 9

4.2 Latency compensation 9

4.3 Slew-rate limit 10

4.4 State machine 10

4.5 Homing 10

4.6 Safety mechanisms 10

4.7 Tracking performance 11

5. Image processing 13

5.1 Table frame: ArUco markers 13

5.2 Homography: pixel → table coordinates 13

5.3 White dot detection 14

5.4 Stability and error sources 15

6. Data collection 16

6.1 Episode cycle 16

6.2 Recorded files 16

6.3 Collected data 17

6.4 Cleaning criteria 18

7. Data analysis 19

7.1 Touch and approach poses 19

7.2 Joint trajectories 19

7.3 Dot position and touch pose 20

7.4 The roll joint is constant 20

8. Modelling 21

8.1 Problem definition 21

8.2 Redundancy of the arm 21

8.3 Candidate models 22

8.4 Validation: leave-one-out (LOO) 23

8.5 Results 23

8.6 Error structure analysis 25

8.7 Estimating the end-point error 25
8.8 Model surface 26
9. Trajectory planning 27
9.1 Approach pose 27
9.2 Minimum-jerk trajectory 27
9.3 Motion phases 28
9.4 Safety layers 28
10. Deployment on the real arm 29
10.1 Run flow 29
10.2 Runs 29
10.3 Evaluation 30
11. Limitations and possible improvements 31
12. Conclusion 31
Appendix A. Symbols 32
Appendix B. File map and commands 33
References 34

1. Introduction

1.1 Goal and scope

The goal of the project is to build a **leader–follower (teleoperation)** robot arm system, to record human demonstrations of a task with it — touching a white dot on the table — and to derive from these recordings a model that can do the task on its own. The hardware (two arms) was built by the user from Dynamixel AX-12A servos.

Every step is explained in this report together with the mathematics behind it: servo units, leader–follower mapping and latency compensation, table coordinates from the camera (homography), dot detection, data analysis, regression models and validation, the redundancy of the arm, minimum-jerk trajectory planning and deployment on the real arm. The appendices give a map of the code and the parameters used.

1.2 Overall approach

1. **Hardware and firmware:** two arms and a leader–follower program running on the OpenCM9.04; data stream, button and PC control mode.
2. **Vision:** homography from four ArUco markers at the table corners; detecting the white dot and converting it to table coordinates.
3. **Data collection:** the arms go to the home pose, the dot is placed, the leader is guided by hand onto the dot; an episode ends at the moment of touch.
4. **Analysis and model:** cleaning the episodes, extracting the touch pose, (u, v) → pose regression and validation.
5. **Deployment:** moving to the predicted pose with a minimum-jerk trajectory and touching; safety layers.

> **Design decision: why a simple model?**
>
> The task is **single-step and static**: the dot does not move and the arm only has to reach it. In that case, instead of imitating the whole trajectory, it is enough to predict the *final pose* from the dot position and to draw the path in between ourselves (minimum-jerk). With 114 samples, a degree-3 polynomial gave better results than a neural network (Section 8).

2. System architecture

2.1 Hardware

Table 2. *Hardware components*

Component	Details
Robot arms	2 arms with 5 axes each: base, shoulder, elbow, wrist, roll (wrist rotation). 1 × Dynamixel AX-12A per joint
Controller	OpenCM9.04 (STM32 based); Dynamixel Protocol 1.0; 1 Mbps; both arms on the same bus (unique IDs)
Camera	Phone (Iriun Webcam), looking down at the table, 1280 × 720
Table frame	4 ArUco markers at the table corners (DICT_4X4_50, ID 0–3), about 5 cm
Computer	Ubuntu, Python 3.12; OpenCV 4.6; NVIDIA GTX 1650 Ti (4 GB) — training was done on the CPU
Power	Battery (12 V class); a 12 V 30 A power supply was also found while this report was written
Button	User button on the OpenCM; pin 16 for an external button (pin → GND)

Table 3. *Servo IDs (bottom to top)*

	Base	Shoulder	Elbow	Wrist	Roll
Leader (moved by hand, torque off)	13	17	14	9	10
Follower (follows the leader)	11	18	16	12	8

2.2 Software components and data flow

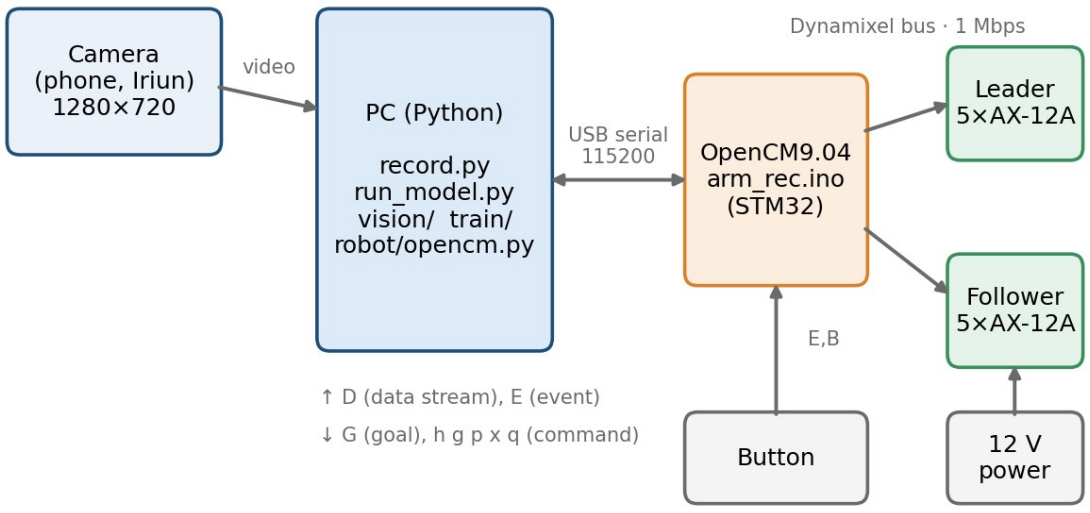


Figure 1. *System block diagram. The camera image goes to the PC; the PC talks to the OpenCM9.04 over a USB serial link; the OpenCM drives both arms over a shared Dynamixel bus.*

The system has three layers:

- **OpenCM9.04** (`arm_rec.ino`): reads and drives the servos, performs the leader–follower mapping, applies the safety checks, streams data to the PC and handles commands from the PC.
- **PC-side Python:** `robot/opencm.py` reads the serial stream and sends commands; `vision/` does the homography and dot detection; `tools/record.py` records the data; `train/` does analysis and modelling; `tools/run_model.py` runs the model on the real arm.
- **Helper tools:** `servo_lab.ino` (a console for inspecting individual servos: scan, info, live monitoring, joint pairing, home/limit settings) and `tools/joint_range.py` (joint range and dead-zone check).

The OpenCM sends one data line per loop and the PC reads it with a background thread:

Table 4. *Serial protocol*

Direction	Line	Meaning
OpenCM → PC	`D,t_ms,state,l1..l5,g1..g5,f1..f5,y1..y5,volt,tmax,tid`	leader position, goal sent to the follower, follower position, follower load, lowest voltage, hottest servo (temperature, ID)
OpenCM → PC	`E,B`	button pressed
OpenCM → PC	`# …`	message for humans
PC → OpenCM	`h g p x q r ?`	home, follow, PC mode, stop, torque off, stream on/off, status
PC → OpenCM	`G,a,b,c,d,e`	follower goals in PC mode (0–1023)

The data line is streamed over 115200 baud USB serial (the OpenCM's USB is native, so this rate is not a bottleneck). The loop rate is set by the read time of the Dynamixel bus; during data collection **≈ 100 Hz** was measured (median interval between samples 9.1 ms, 95% within 14.8 ms).

3. Servo basics

3.1 Units

The position register of the AX-12A ranges from 0 to 1023, which corresponds to 0–300°:

$$\text{degrees} = p \cdot \frac{300}{1023} \tag{1}$$

So one unit is about 0.293°; 512 = 150° (the middle of the servo). All joint values in this report are in these **servo units**. Multiply by 0.293 to convert units to degrees.

Table 5. *AX-12A quantities*

Quantity	Range / unit	Note
Position	0–1023 ↔ 0–300°	1 unit ≈ 0.293°
Speed (moving speed)	1 unit ≈ 0.111 rpm	**0 = unlimited speed**; 0 is never written
Load	10-bit magnitude + direction bit	converted to a signed value (−1023…1023)
Voltage	0.1 V units	120 = 12.0 V; operating range 9–12 V
Temperature	°C	the servo's own limit is 70 °C

3.2 Dead zone

The position sensor reads 0–300°; 300° to 360° (i.e. between 1023 and 0) is a **dead zone**: the position cannot be read and the servo cannot be controlled there. If a joint's working range in the task gets close to 0 or 1023, the reading jumps suddenly and the arm moves unexpectedly. Therefore (i) the home pose was chosen so that the joints are near the middle of the servo range (512); (ii) the firmware treats jumps larger than 100 units in one loop as invalid; (iii) the tools flag values closer than 30 units to 0 or 1023. The real fix for a joint that enters the dead zone is to rotate the servo/horn mounting.

3.3 Power and temperature

The AX-12A operates between 9 and 12 V. The most important practical lesson of the project was that **the power supply determines data quality**: in the first five episodes, recorded while the battery was nearly empty, the lowest servo voltage dropped to about 6.5 V; at this voltage some servos stopped responding to commands (for example servo 14) and the system behaved unreliably. These five episodes were removed from the data set (Figure 8). For the same reason, the recorder continuously monitors the voltage and the temperature of the hottest servo (Section 6).

4. Leader–follower control

The leader arm is moved by hand (its torque is off); on every loop the firmware reads the leader's joint positions and sends them to the follower as goals. This section describes the mathematics of the mapping, the latency compensation and the safety checks.

4.1 Joint mapping

For the leader position l_i of joint i, the follower counterpart is computed as:

$$b_i = H_i^F + s_i \left(l_i - H_i^L \right) \tag{2}$$

Here H^L_i and H^F_i are the **home** values of the two arms, and $s_i \in \{+1, -1\}$ is the mounting direction sign of the two servos (current: {+1, +1, −1, −1, +1}). The mapping is **relative**: the follower deviates by as much as the leader deviates from home. Therefore the absolute offsets of the two arms need not be equal and there is no separate calibration step; the home pose is the “same pose” reference of both arms.

Table 6. *Home values (servo units)*

	Base	Shoulder	Elbow	Wrist	Roll
Leader H^L	512	512	512	512	665
Follower H^F	512	512	512	512	365
Follower safe range m…M	100…923	100…923	100…923	100…923	100…923

The home pose was chosen with the arms standing upright; the roll has an offset of 45° (≈ 153 units) (512 + 153 = 665 on the leader, 512 − 147 = 365 on the follower).

4.2 Latency compensation

The servo cannot follow a command instantly; there is a latency of about 60 ms. To reduce it, the goal is shifted slightly ahead according to the leader's velocity. First the velocity of the goal is estimated with a low-pass filter:

$$\hat{v}_i[k] = (1 - \alpha)\, \hat{v}_i[k-1] + \alpha \frac{b_i[k] - b_i[k-1]}{\Delta t} \tag{3}$$

Here α = 0.3 is the filter coefficient and Δt is the loop period. The lead term and the final goal:

$$\ell_i = \mathrm{clip}(T_\ell\, \hat{v}_i,\ -\ell_{\max},\ +\ell_{\max}) \tag{4}$$

$$\tilde{g}_i = \mathrm{clip}(b_i + \ell_i,\ m_i,\ M_i) \tag{5}$$

T_ℓ = 60 ms and ℓ_{max} = 30 units (≈ 8.8°). Since the velocity estimate is based on a difference, the 1-unit quantization of the servo position amplifies noise: a 1-unit difference in 10 ms is 100 units/s, i.e. a 6-unit lead offset. This is why the goal of the roll joint jitters by ±5 units (Figure 3).

4.3 Slew-rate limit

The goal itself cannot jump instantly; on every loop it advances by at most $R\Delta t$:

$$g_i[k] = g_i[k-1] + \mathrm{clip}(\tilde{g}_i - g_i[k-1],\ -R\,\Delta t,\ +R\,\Delta t) \tag{6}$$

Table 7. *Slew-rate limit R by mode*

Mode	R [units/s]	Angular rate	Purpose
Sync (start)	40	≈ 12 °/s	the follower approaches the leader slowly; following starts once the error is ≤ 6 units on all joints
Follow (teleop)	1200	≈ 352 °/s	only a protection against corrupt reads
PC mode (model)	300	≈ 88 °/s	upper bound for the goals sent by the model

4.4 State machine

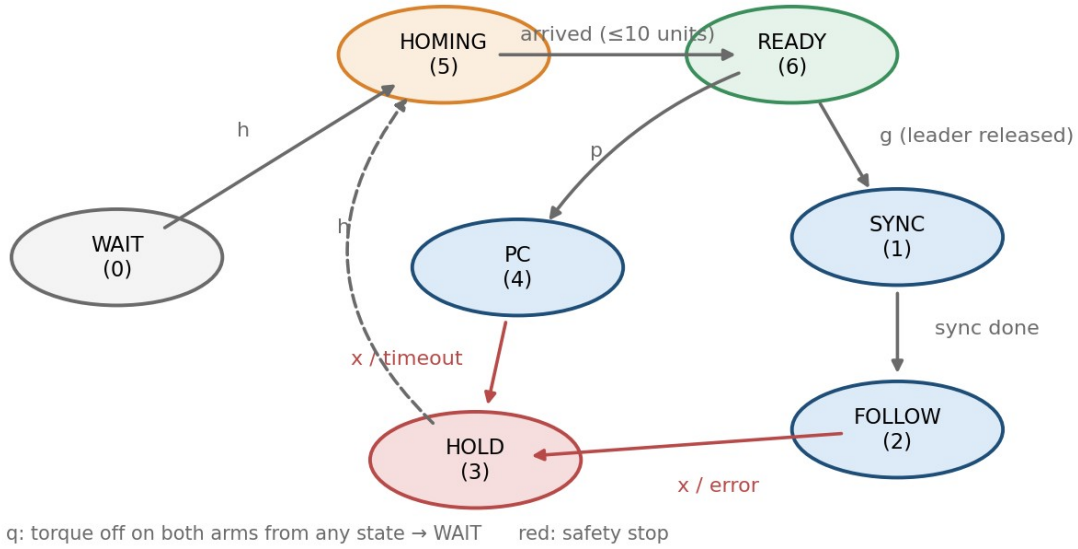


Figure 2. *Firmware state machine. The numbers in parentheses are the state codes in the data line.*

The normal flow is **WAIT** → **HOMING** → **READY**. In READY both arms are held at home with torque. During data collection `g` releases the leader and starts following; when running the model, `p` enables PC mode. On any error the follower goes into the **HOLD** state (holds its current position with torque on).

4.5 Homing

When the home command arrives, every servo first gets torque at its current position (to avoid a jump), then the goal is set to home and the servo's own speed limit (80 units ≈ 8.9 rpm) moves both arms slowly. Once |position − home| ≤ 10 units on all joints the state becomes READY; if home is not reached within 10 s the state falls back to HOLD.

4.6 Safety mechanisms

$$|g_i - f_i| > 80 \text{ units} \quad \text{for} \quad t > 500 \text{ ms} \Rightarrow \text{HOLD} \tag{7}$$

Table 8. Firmware safety checks

Check	Threshold	Result
Tracking error |g − f|	> 80 units for > 500 ms	follower freezes (HOLD)
Leader jump (dead zone)	> 100 units in one loop	that reading is rejected; HOLD after 10 consecutive errors
Read error	10 consecutive failures	HOLD
No commands from the PC (PC mode)	300 ms	HOLD
Voltage	< 6.4 V	torque off on both arms, program stops
Temperature	> 60 °C	torque off on both arms, program stops

4.7 Tracking performance

How well the follower tracks its goal can be measured directly from the recorded data. Figure 3 shows the leader (converted to follower units), the goal sent and the measured follower position in one episode; Figure 4 gives the distribution of |goal − measured position| over all episodes.

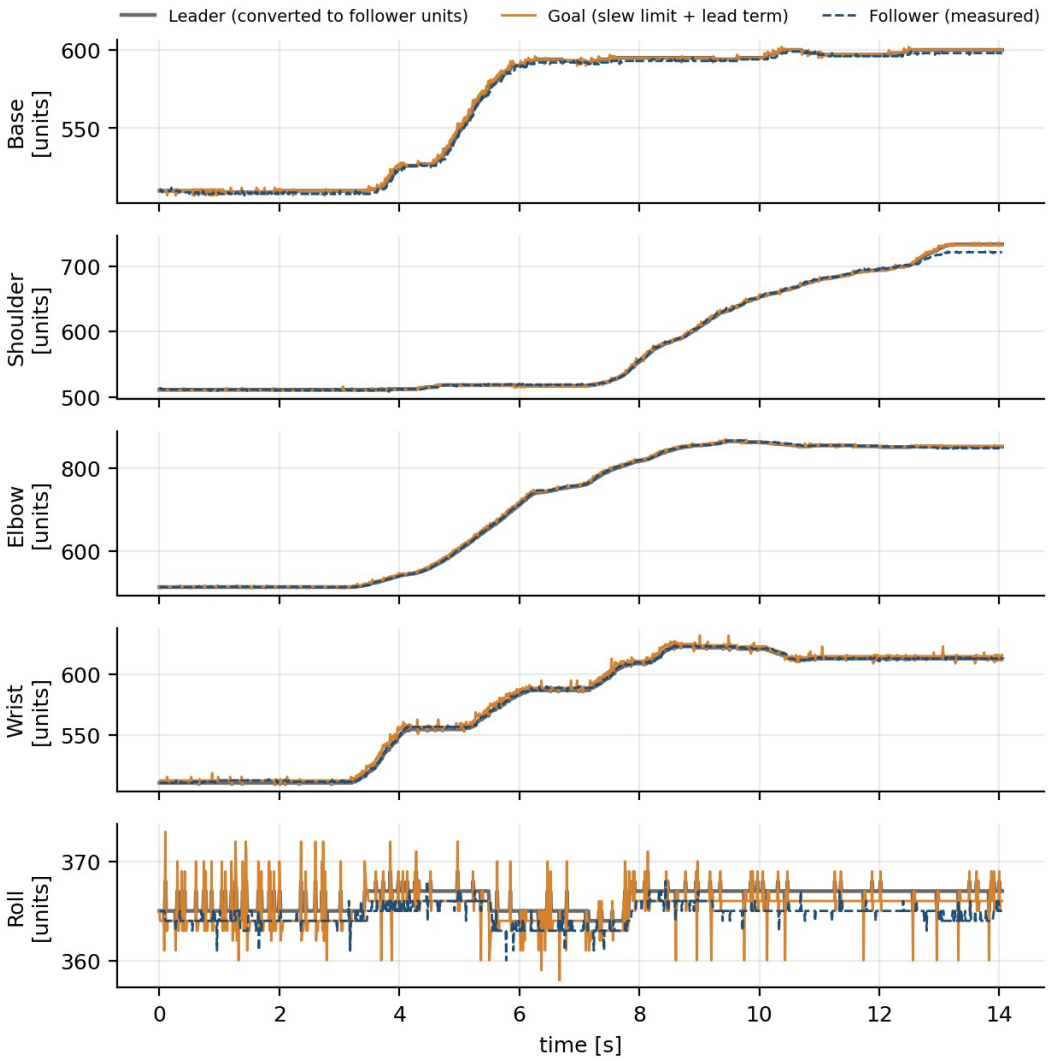


Figure 3. *Leader, goal and follower in an example episode (ep_0060). Leader values are converted to follower units: $F_0 + s \cdot (l - L_0)$. The jitter in the roll panel is quantization noise in the velocity estimate.*

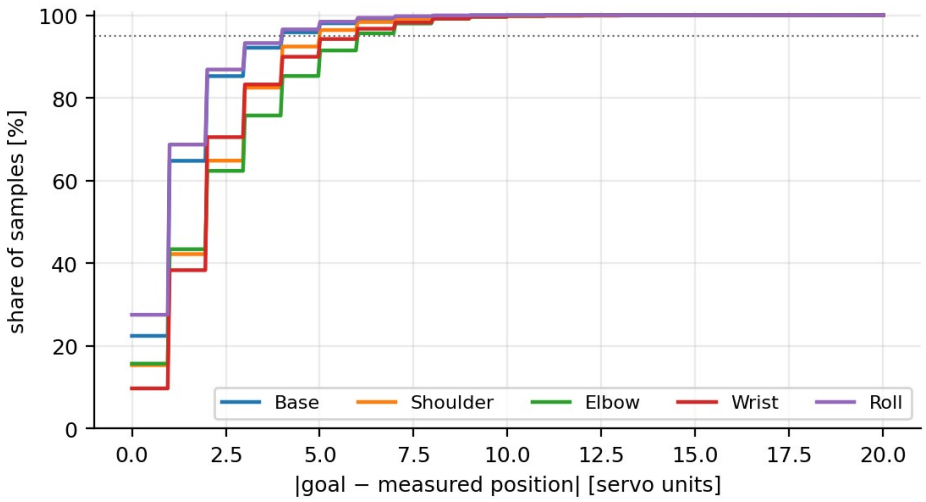


Figure 4. *Cumulative distribution (CDF) of the absolute difference between the goal and the measured follower position over all used episodes.*

Table 9. *Goal – measured position difference (servo units), 172,814 samples*

Joint	Median	95%	99%	Max
Base	1	4	6	279
Shoulder	2	5	7	18
Elbow	2	6	8	277
Wrist	2	6	8	15
Roll	1	4	6	12

The median difference is 1–2 units (≈ 0.3–0.6°) and the 95th percentile is 4–6 units (≈ 1.2–1.8°). So the follower tracks the leader closely and **the recorded follower position is the motion that was actually performed**. Since the median goal–position difference at the moment of touch is also 0–2 units, the measured positions the model uses as targets and the commanded goals are practically equivalent.

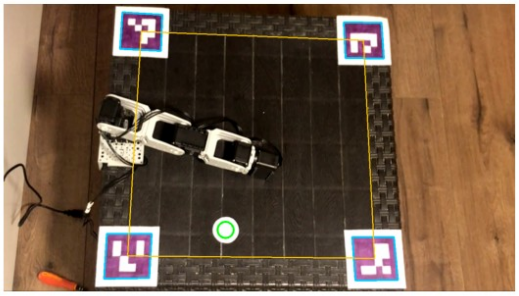
5. Image processing

The position of the dot on the table is measured from the camera only, independently of the coordinates the arm uses. The camera sees the table from above but at an angle; a pixel position is not directly a table position. Therefore a transformation is first built from the markers at the table corners.

5.1 Table frame: ArUco markers

ArUco markers (4×4-bit dictionary, ID 0–3) are stuck on the four corners of the table: ID0 top left, ID1 top right, ID2 bottom right, ID3 bottom left (as seen from the camera). The marker corners are found with OpenCV and the **center** of each marker (the mean of its 4 corners) is used. If the arm partly covers a marker, its last known center is used; the table is defined once all four markers have been seen at least once. Because the markers are detected again in every frame, the transformation updates itself if the camera shifts slightly.

(a) Markers, table and detected dot



(b) Mask after white threshold and opening

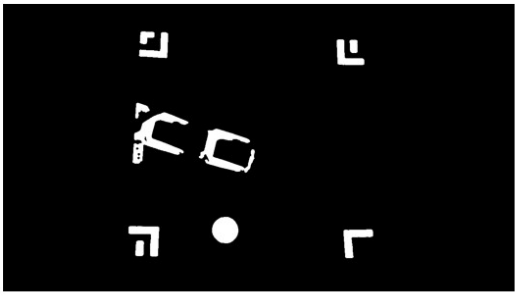


Figure 5. *Camera image. (a) markers, the table quadrilateral formed by the marker centers, and the detected dot; (b) mask after the white threshold and morphological opening (arm parts and the white parts of the markers also pass; they are removed by the area and circularity filters).*

5.2 Homography: pixel → table coordinates

The relation between two views of points on a plane is expressed by a **homography**. The pixel coordinate *(x, y)* in homogeneous form is multiplied by a 3×3 matrix **H** and divided by the last component:

$$(u',\, v',\, w')^{\top} = \mathbf{H}\,(x,\, y,\, 1)^{\top}, \qquad u = \frac{u'}{w'},\;\; v = \frac{v'}{w'} \tag{8}$$

H is only defined up to scale, so it has 8 free parameters (h_{33} = 1). Each point correspondence gives two linear equations; moving the denominator to the other side:

$$u\,(h_{31}x + h_{32}y + h_{33}) = h_{11}x + h_{12}y + h_{13} \tag{9}$$

$$v\,(h_{31}x + h_{32}y + h_{33}) = h_{21}x + h_{22}y + h_{23} \tag{10}$$

Four correspondences (four marker centers) give 8 equations and the linear system in 8 unknowns is solved exactly (`cv2.getPerspectiveTransform`). The marker centers are mapped to the corners of the unit square: ID0 → (0,0), ID1 → (1,0), ID2 → (1,1), ID3 → (0,1). The result is the *(u, v)* coordinate of the dot: *u* runs left to right and *v* top to bottom, both in 0–1 (Figure 6).

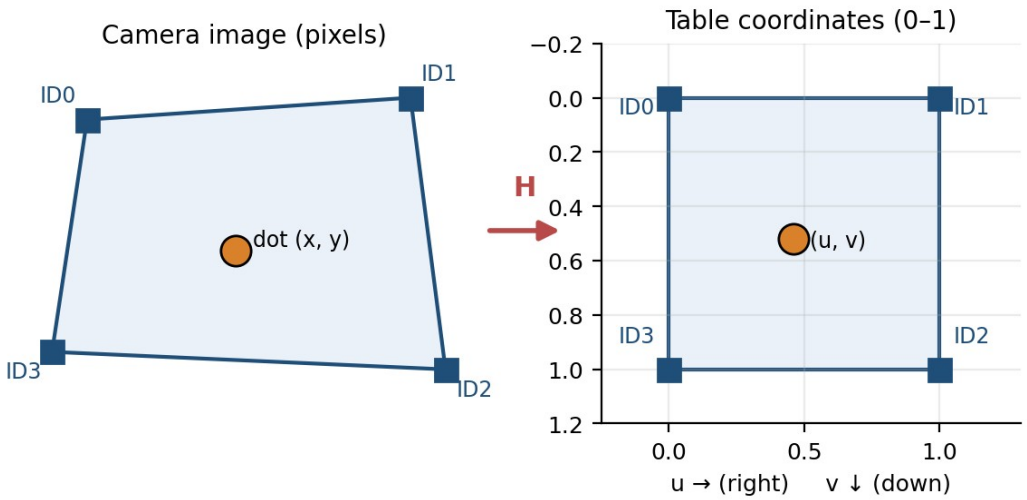


Figure 6. *Homography: the (skewed) quadrilateral in the camera image is mapped to the unit square using the marker centers.*

Assumptions and limits

• The dot lies in the plane defined by the marker centers (the table surface); if it is raised above the surface the homography is wrong.

• The coordinate is **dimensionless**: the 0–1 range is scaled by the distance between the marker centers, not millimetres.

• Lens distortion is not corrected.

• If two of the four markers are covered, the last known centers are used; the markers must be stuck firmly to the table.

5.3 White dot detection

The dot is found in the following order:

1. The image is blurred with a 5×5 Gaussian filter and converted to HSV.
2. **White threshold:** white means low saturation and high brightness: $S \le 60$ and $V \ge 170$.
3. Only the inside of the table quadrilateral is considered (cuts out the effect of the arm and the area outside the table).
4. **Morphological opening** (7×7 elliptical structuring element): removes thin table lines and noise.
5. Area A and **circularity** are computed for each component; those with $300 \le A \le 8000$ pixels and $C \ge 0.70$ are candidates.
6. The centroid of the largest candidate is taken and converted to table coordinates with the homography.

$$C = \frac{4\pi A}{P^2} \le 1, \qquad C \ge 0.70 \tag{11}$$

Circularity equals 1 for a perfect circle (*A* area, *P* perimeter). Arm parts and cables are long and irregular and score low, so white but non-round objects are rejected. The center is found from the geometric moments of the component (sub-pixel precision):

$$c_x = \frac{m_{10}}{m_{00}}, \qquad c_y = \frac{m_{01}}{m_{00}} \tag{12}$$

Table 10. *Dot detection parameters*

Parameter	Value
White threshold	S ≤ 60, V ≥ 170 (HSV)
Morphological opening	7 × 7 ellipse
Area range	300–8000 pixels
Circularity	C ≥ 0.70
Stability	median of the last 15 frames; (u, v) standard deviation < 0.01

5.4 Stability and error sources

During data collection and when running the model, the **median of the last 15 frames** is used instead of a single frame; if the (u, v) standard deviation is above 0.01 the dot counts as “unstable” and no episode is started. The main error sources are lighting changes (which affect the threshold), the dot being covered by the arm or a hand, markers shifting, and the table-plane assumption.

6. Data collection

6.1 Episode cycle

The data was collected with `tools/record.py`. An episode consists of the following steps:

1. **Home:** both arms move slowly to the upright pose; the leader is also held with torque so it does not fall over. Meanwhile the dot is placed at a new position on the table.
2. **Start (SPACE or button):** the program measures the dot with the camera and stores it; the leader is released, the follower follows it, recording starts. Starting is blocked if the dot is not visible/unstable or a voltage or temperature threshold is exceeded.
3. **Touch:** the leader is guided by hand to touch the dot lightly.
4. **Stop (SPACE/button):** recording ends and both arms return home. Because **recording ends at the moment of touch**, the last sample of each episode is the touch pose; the return motion is not recorded.
5. **Confirm:** ENTER/button saves, `d` discards. Then the cycle starts again.

On screen the table is divided into 4×4 cells and each cell shows how many episodes were collected in that region; this way uncovered regions (red 0) can be seen and the dot placed there. The battery voltage and the temperature of the hottest servo are shown as well.

Table 11. *Voltage and temperature thresholds (recorder)*

Quantity	Warning (yellow)	No new episode (red)	Firmware emergency stop
Battery voltage	< 11.0 V	< 10.5 V	< 6.4 V
Hottest servo	≥ 50 °C	≥ 55 °C	> 60 °C

6.2 Recorded files

Table 12. *Files per episode (`data/episodes/ep_NNNN/`)*

File	Contents
`data.npz`	`t_pc, t_ms, state, leader, goal, follower, load, volt, temp, frame_t` (≈ 100 Hz)
`video.mp4`	raw camera footage; matched to the serial data via `frame_t`
`meta.json`	dot (u, v) and pixel position, homography **H**, marker positions, duration, start/end voltage and temperature

Serial data and video are matched using the PC clock (`t_pc` and `frame_t`). Since the dot position is constant during an episode it is measured once at the start (before the arm covers the dot).

6.3 Collected data

Table 13. *Summary of the collected data*

Quantity	Value
Recorded episodes	119
Used episodes	114 (the first 5 were dropped because the voltage was < 9 V)
Used samples	172,814
Sampling	≈ 100 Hz
Episode duration	median 15.0 s; 8.8–26.8 s
Lowest voltage per episode (used ones)	10.4–11.0 V (median 10.7 V)
Dot range	u: 0.07–0.53, v: 0.10–0.93
Disk size	≈ 0.98 GB (mostly video)

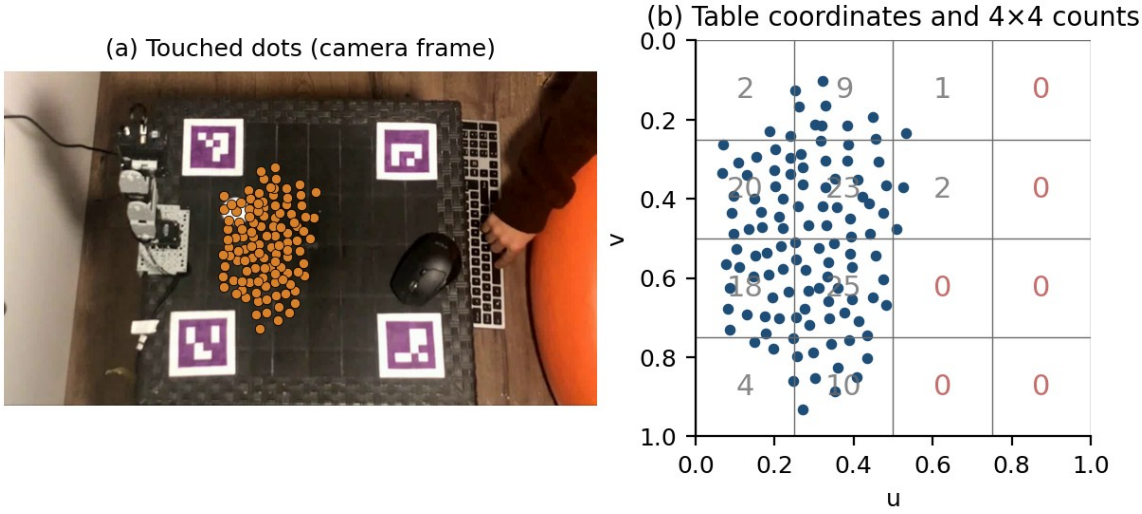


Figure 7. *(a) Touched dots, drawn on the camera frame of one episode with the inverse homography. (b) The same dots in table coordinates with 4×4 cell counts. The dots cover the reach area of the arm (on the left); the right half is out of reach.*

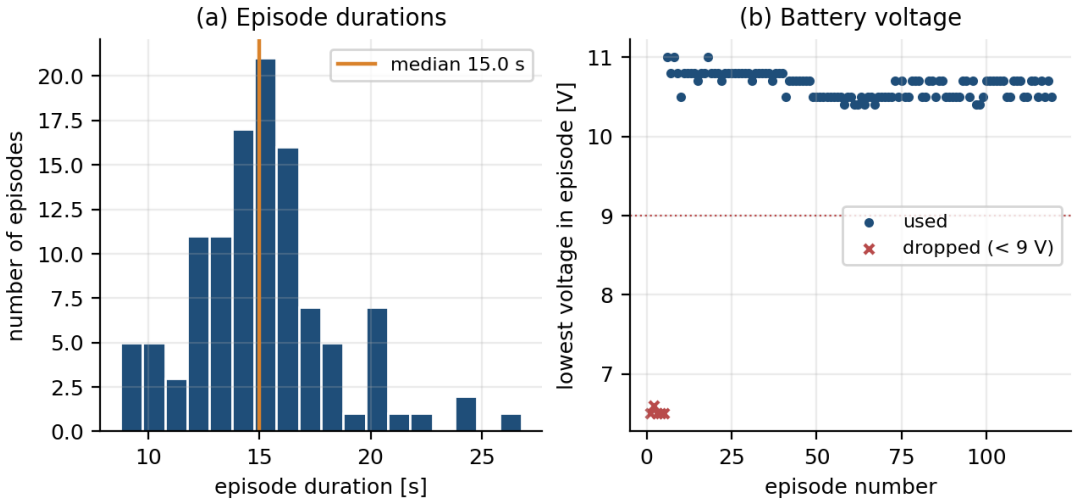


Figure 8. *(a) Episode durations. (b) Lowest servo voltage measured in each episode; the first five episodes were recorded on a nearly empty battery and dropped.*

The dots are spread fairly evenly over the reach area in the left half: in the 4×4 count the middle cells contain 18–25 episodes and the edge cells 1–4. This means the model will be better in the middle than at the edges.

6.4 Cleaning criteria

Table 14. *Episode drop criteria (`train/prepare.py`)*

Criterion	Threshold
Lowest voltage	dropped if < 9.0 V
Episode with a warning	dropped if the follower froze during recording
Duration	dropped if outside 4–40 s
Motion at the touch point	dropped if the joint standard deviation over the last 0.3 s is > 6 units

7. Data analysis

7.1 Touch and approach poses

Because an episode ends at the moment of touch, the **touch pose** is computed from the last samples of the episode (the median is robust to noise):

```
touch_pose = median( follower position , last 0.3 s )            # 5 joints
pre_pose   = median( follower position , 3.0 s before the touch )
pre_delta  = median( pre_pose − touch_pose )   (all episodes)   # a single vector
```

The start pose is very similar in all episodes (standard deviation below 1 unit); this shows that the home system works consistently.

7.2 Joint trajectories

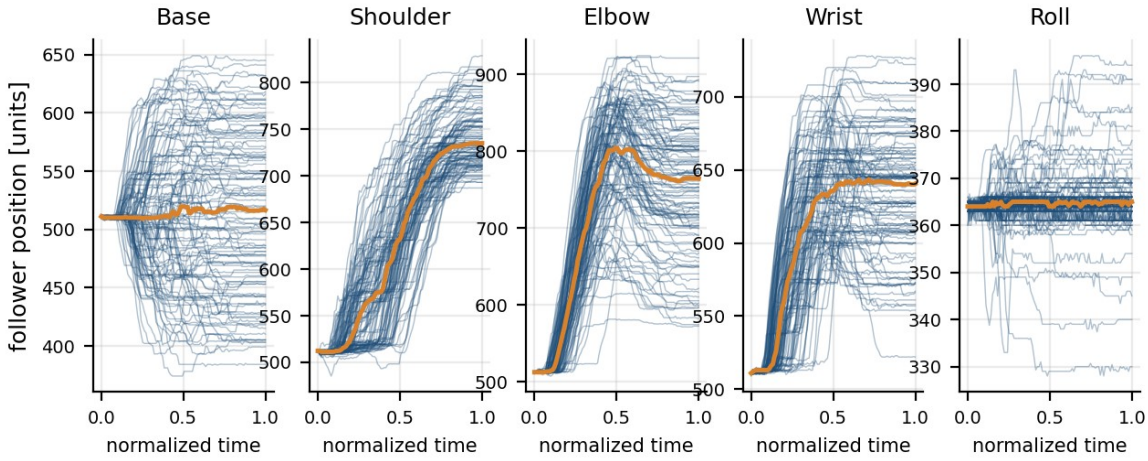


Figure 9. *Follower joint trajectories of all episodes in normalized time (0 = start, 1 = touch). The orange line is the median.*

- The elbow and wrist move first (10–40% of normalized time), the shoulder later; the timing of the motions varies noticeably between episodes.
- The motion of the base joint splits into two directions depending on the dot (dot on the left or on the right).
- The roll trajectories form an almost flat line.

This large variability in timing is the reason for predicting **only the final pose** and drawing the path in between ourselves, instead of imitating the trajectory exactly.

7.3 Dot position and touch pose

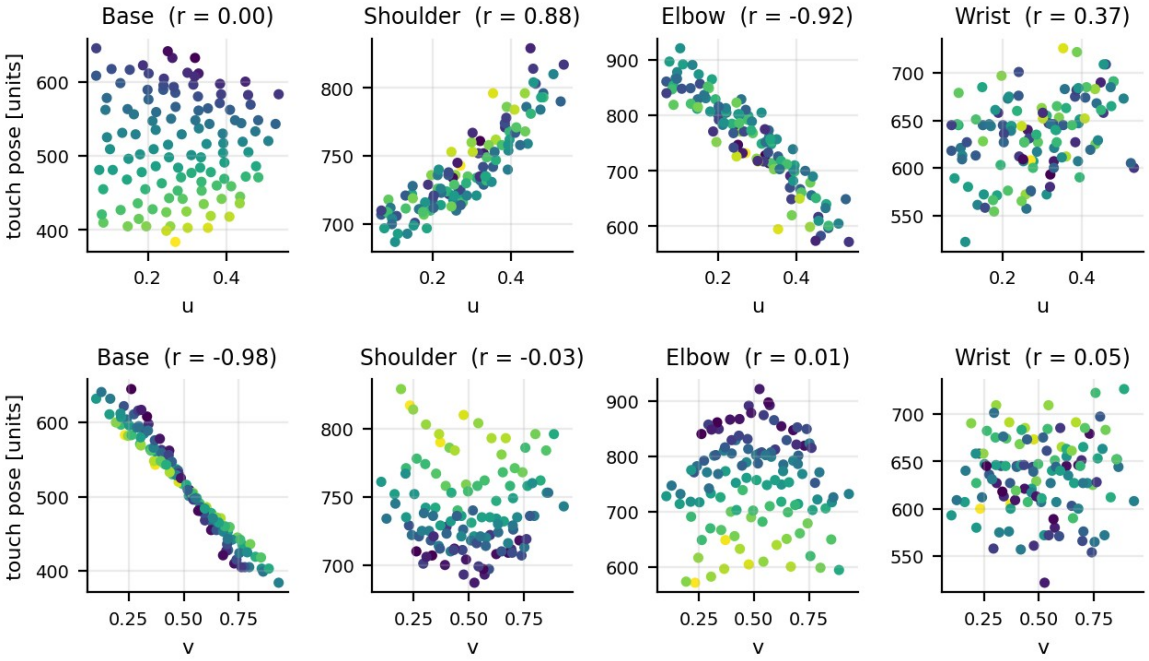


Figure 10. *Relation between the touch pose and the dot coordinate (u: top row, v: bottom row). Color shows the other coordinate. r in the title is the Pearson correlation.*

Table 15. *Pearson correlation (touch pose — dot coordinate)*

Joint	with u	with v
Base	0.00	-0.98
Shoulder	0.88	-0.03
Elbow	-0.92	0.01
Wrist	0.37	0.05

The measured relations agree with the geometry of the arm: the **base** joint determines which direction the arm turns to and has an almost linear relation with *v* (r = -0.98). The **shoulder** and **elbow** determine how far the arm reaches and are strongly related to *u* (r = 0.88 and -0.92). The **wrist** joint is only weakly related to the dot position and is scattered; the reason is the redundancy explained in the next section.

7.4 The roll joint is constant

The roll (wrist rotation) plays no role in this task: its value in the touch poses stays around the home value (365) in nearly every episode; the deviations are small wobbles from involuntary hand motion. Therefore roll is not part of the model and is kept constant at its home value.

8. Modelling

8.1 Problem definition

The input is the table coordinate of the dot *(u, v)*; the output is the four joint angles at the moment of touch, **q** = (q_{base}, $q_{shoulder}$, q_{elbow}, q_{wrist}) (in servo units). Roll is kept constant. The training set is 114 pairs from 114 episodes: {(u_i, v_i) → $\mathbf{q}_i$}.

8.2 Redundancy of the arm

The end point of a planar three-link arm (shoulder, elbow, wrist) touching the table is written in terms of the joint angles $\theta_{1...3}$ and the link lengths $l_{1...3}$ as:

$$r(\theta) = \sum_{k=1}^{3} l_k \cos \phi_k, \qquad z(\theta) = \sum_{k=1}^{3} l_k \sin \phi_k, \qquad \phi_k = \sum_{j=1}^{k} \theta_j \tag{13}$$

Here *r* is the horizontal distance of the end point from the base, *z* its height and φ_k the absolute angle of link k. Touching a point on the table means **two conditions**: the distance must equal the target value and the height must be the table surface. There are two equations for three unknowns ($\theta_{1...3}$):

$$r(\theta) = r^*, \quad z(\theta) = 0, \qquad \mathbf{J} = \frac{\partial(r, z)}{\partial \theta} \ (2 \times 3), \quad \dim \ker \mathbf{J} = 3 - 2 = 1 \tag{14}$$

The null space of the Jacobian is 3 − 2 = **1-dimensional**: there are infinitely many poses that reach the same point; they are all distinguished by a single parameter (for example the approach angle of the last link) (Figure 11). Since we lowered the arm onto the dot holding the wrist slightly differently each time, the shoulder–elbow–wrist combination varies between episodes while the end point lands in the same place.

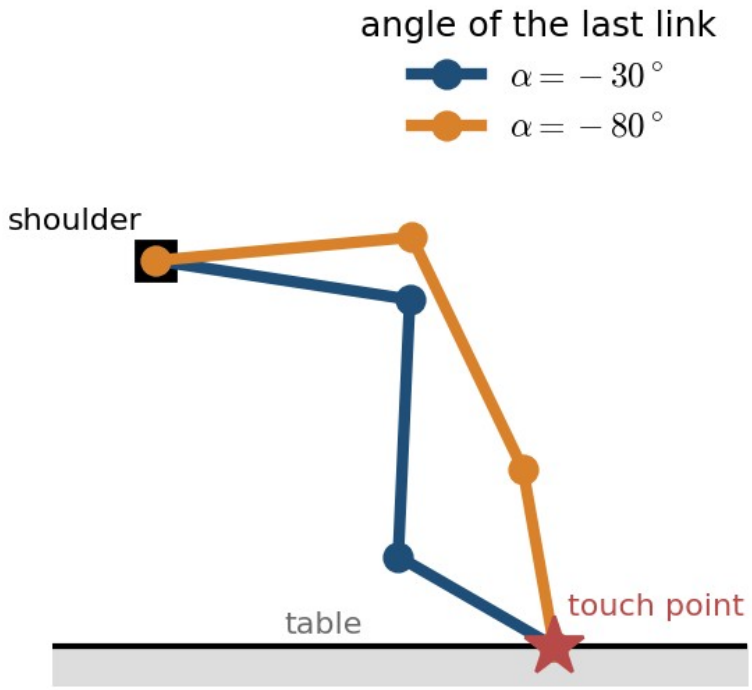


Figure 11. *Redundancy (schematic; the link lengths are not those of the real arm): two poses reaching the same touch point with two different angles of the last link.*

> **Consequence**
>
> For a given *(u, v)* the correct poses form a **set**, not a single point. A single-valued regression model predicts the "mean" of this set. If the set is roughly a straight line, the mean stays inside the set and is a valid pose; if the set is curved, the mean may be invalid. The analysis below shows that the set is approximately linear within this data range.

8.3 Candidate models

Polynomial ridge regression (the model used). Degree-3 polynomial feature vector (10 features):

$$\varphi(u,v) = [\,1,\ u,\ v,\ u^2,\ uv,\ v^2,\ u^3,\ u^2v,\ uv^2,\ v^3\,]^\top \tag{15}$$

The features are stacked into a matrix **X** (one row per episode) and the targets into **Y**. The ridge (Tikhonov) solution is in closed form:

$$\mathbf{W} = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{Y}, \qquad \lambda = 10^{-3} \tag{16}$$

$$\hat{\mathbf{q}}(u,v) = \mathbf{W}^\top \varphi(u,v) \tag{17}$$

$\lambda = 10^{-3}$ keeps the solution stable by damping the near-linear dependence between the features. **W** is a 10×4 weight matrix; the whole model consists of 40 numbers.

GP / RBF kernel ridge — in addition to a linear base, it models the residuals with a kernel-based function:

$$k(\mathbf{a},\mathbf{b}) = \exp\left(-\frac{\|\mathbf{a}-\mathbf{b}\|^2}{2\,\ell^2}\right) \tag{18}$$

ℓ = 0.4 (length scale) and a regularization of 0.01 were chosen. **k-nearest neighbours** (k = 5):

$$\hat{\mathbf{q}} = \frac{\sum_{n\in\mathcal{N}_k} w_n\,\mathbf{q}_n}{\sum_{n\in\mathcal{N}_k} w_n}, \qquad w_n = \frac{1}{d_n + 10^{-3}} \tag{19}$$

MLP (neural network): 2 hidden layers × 64 neurons, tanh activation, AdamW (learning rate $3\cdot10^{-3}$, weight decay 10^{-4}), 1500 iterations, standardized targets.

8.4 Validation: leave-one-out (LOO)

Because the training set is small, **LOO** was used instead of a standard train/test split: each time one episode is held out for testing, the model is trained on the remaining 113 episodes and the held-out episode is predicted. Doing this for all episodes means each episode is predicted as an "unseen" point. The error measure is the root-mean-square (RMS) error per joint; $\hat{q}^{(-i)}$ is the prediction of the model trained with episode i left out:

$$\mathrm{RMS}_j = \sqrt{\frac{1}{N}\sum_{i=1}^{N}\left(\hat{q}_{i,j}^{(-i)} - q_{i,j}\right)^2} \tag{20}$$

For the MLP, 10-fold cross-validation was used instead of LOO because training is expensive. The "no model" reference predicts the mean pose for every episode; its RMS equals the standard deviation of that joint.

8.5 Results

Table 16. *LOO error, RMS (servo units; 1 unit ≈ 0.293°)*

Model	Base	Shoulder	Elbow	Wrist
Mean pose (reference)	67.6	31.4	81.8	40.2
Polynomial degree 1	14.3	15.2	33.3	38.3
Polynomial degree 2	6.7	11.3	26.5	39.1
Polynomial degree 3 (used)	4.6	11.5	27.0	39.2
GP / RBF	3.4	11.5	27.1	39.7
k-nearest neighbours (k=5)	7.0	12.4	28.9	39.7
MLP 2×64 (10-fold)	4.7	16.0	40.8	57.9

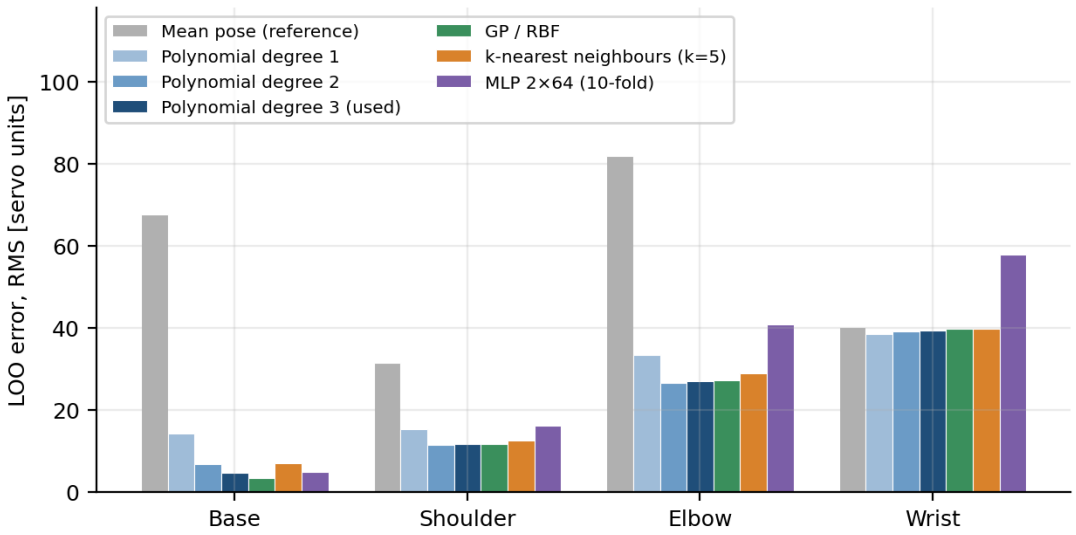


Figure 12. *LOO errors of the models (lower is better). The reference is the error of predicting the mean pose.*

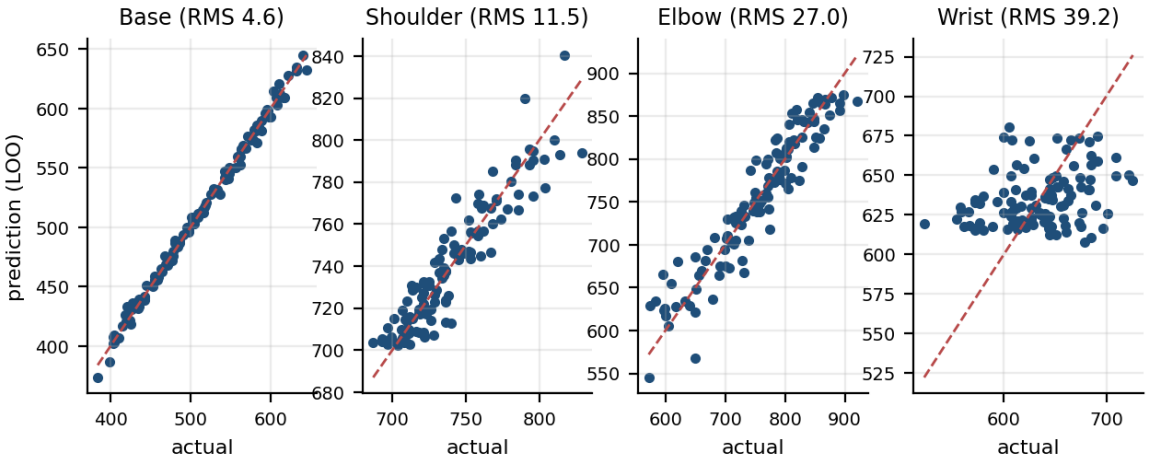


Figure 13. *Degree-3 polynomial: LOO prediction versus the actual touch pose (dashed line: perfect fit).*

- The **base** joint was learned very well with an error of about 4.6 units (≈ 1.4°); a 15-fold improvement over the reference.
- The **shoulder and elbow** errors dropped to roughly a half to a third of the reference.
- The **wrist** joint barely improved; the reason is the redundancy shown in the next section, not a shortcoming of the model.
- The polynomial (degree 3) and the GP gave almost the same result; k-nearest neighbours was slightly behind. The **MLP did worst**: 114 samples are too few for a neural network.

In practice the **degree-3 polynomial ridge** was used: as accurate as the GP, much simpler (40 numbers in total), solved in closed form, and it needs only one matrix product at run time.

8.6 Error structure analysis

Per-joint error is misleading because of redundancy: the shoulder, elbow and wrist compensate for each other. To see this, the covariance of the LOO error vectors of these three joints was examined; since **C** is symmetric positive semi-definite, its eigendecomposition shows in which directions the error is concentrated:

$$\mathbf{C} = \frac{1}{N}\sum_{i=1}^{N} \mathbf{e}_i \mathbf{e}_i^{\top}, \qquad \mathbf{C}\,\mathbf{v}_k = \lambda_k \mathbf{v}_k, \qquad \mathrm{share}_k = \frac{\lambda_k}{\sum_m \lambda_m} \tag{21}$$

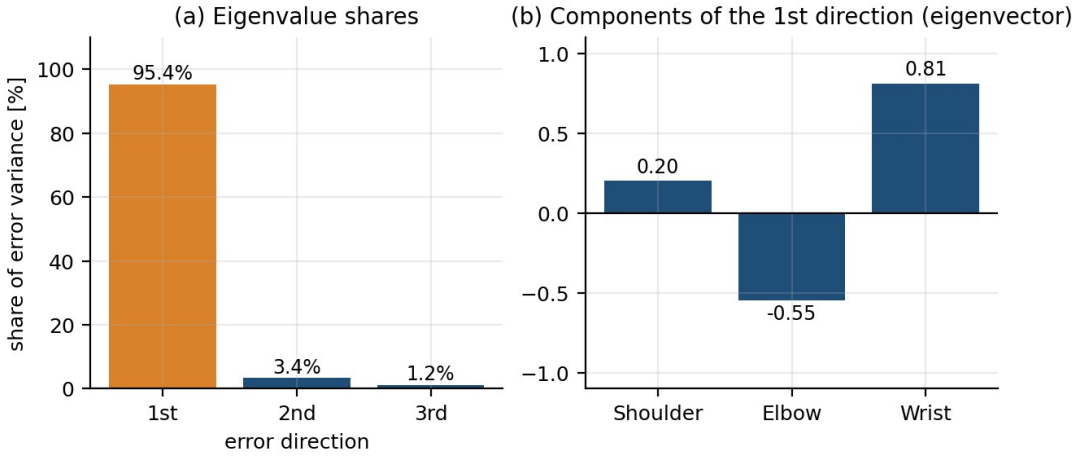


Figure 14. *Eigen-analysis of the LOO error vector (shoulder, elbow, wrist). (a) share of the error variance in each direction; (b) components of the largest direction.*

The result is striking: **95.4%** of the error variance lies in a single direction (shoulder, elbow, wrist components ≈ (0.20, -0.55, 0.81)). This direction is the **redundancy direction** that moves the joints together without changing the end point, i.e. the wrist posture the user chose differently each time. The error in the remaining two directions is small (standard deviations ≈ 9.1 and 5.5 units); together they give **10.6 units ≈ 3.1°** RMS error. This is the error that actually moves the end point.

8.7 Estimating the end-point error

With the small-angle approximation, an error $\delta\theta$ at a joint shifts the end point by about $L \cdot \delta\theta$. The length of the arm from the base to the tip is about 34 cm (measured, not from the joint center); in the worst case the whole error acts at the full distance of the tip:

$$\delta \approx L\,\delta\theta = 34\ \mathrm{cm} \times 0.054\ \mathrm{rad} \approx 1.8\ \mathrm{cm} \tag{22}$$

In reality part of the error is in the near joints and acts with a shorter lever than the whole arm; the base error (≈ 1.4°) corresponds to ≈ 0.4 cm at a radius of about 25 cm. The expected end-point error is therefore around **≈ 1–1.5 cm**. The diameter of the dot was estimated from the image as about 2.5–3 cm; so the error is small enough to land inside the dot in most attempts. The 0.5 cm protrusion of the screw tip is not added separately, because the touch poses were recorded with the screw tip in contact.

> **This is an estimate, not a measurement**
>
> The error in centimetres was calculated geometrically from the angle error. The distance of the end point from the dot was not measured on the real arm (see Section 11).

8.8 Model surface

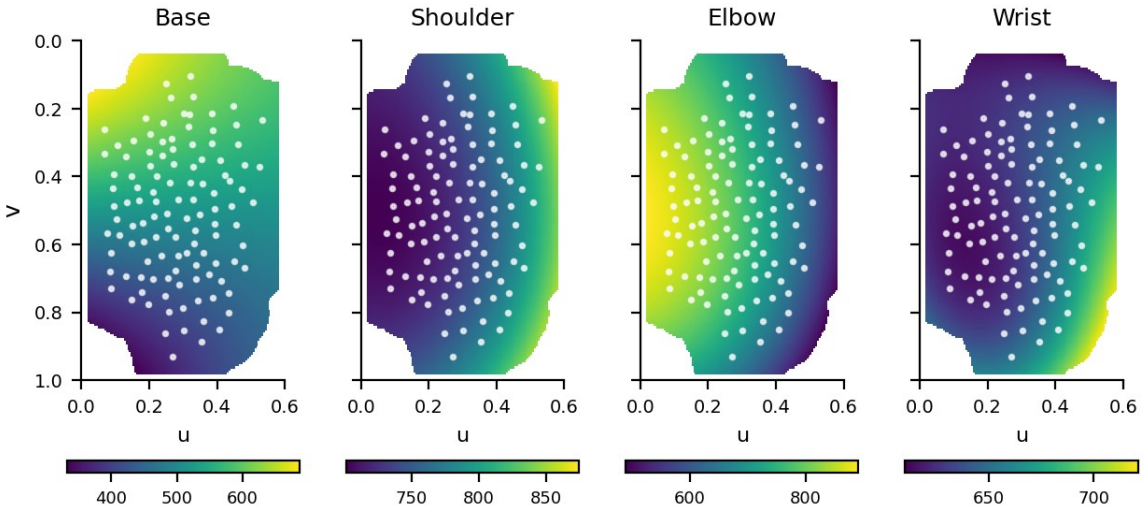


Figure 15. *Prediction of the selected model over the table (u, v → joint). White dots are training points; regions farther than 0.12 from the nearest training point are considered unreliable and hidden.*

The prediction surfaces look smooth and monotonic; this shows that the polynomial model does not overfit the data. Outside the training points (especially with a degree-3 polynomial) predictions can degrade quickly; this is why the run script rejects dots farther than 0.12 table units from the nearest training point.

9. Trajectory planning

9.1 Approach pose

No clear increase in load or in the goal–position difference was observed at the moment of touch (the touch was very light); therefore lowering the arm by *pressing* it onto the table and detecting contact from the load is not reliable. Instead, the human approach is imitated: the arm first goes to an **approach pose** just above the table and then descends slowly to the touch pose:

$$\mathbf{q}_{\text{approach}} = \hat{\mathbf{q}}_{\text{touch}} + \delta, \qquad \delta = \text{median}(\mathbf{q}_{3\,\text{s}} - \mathbf{q}_{\text{touch}}) \tag{23}$$

Here **δ** is the per-joint median over all episodes of the difference “pose 3 s before the touch − touch pose”: ≈ (0, -9, 2, 0) units (base, shoulder, elbow, wrist). 3 s was chosen as the lead time: 1 s before the touch the pose difference is ≈ 1 unit (the arm is already still), 3 s before ≈ 10 units (≈ 3°, roughly 1.5–2 cm above), 4 s before ≈ 20 units (Figure 16).

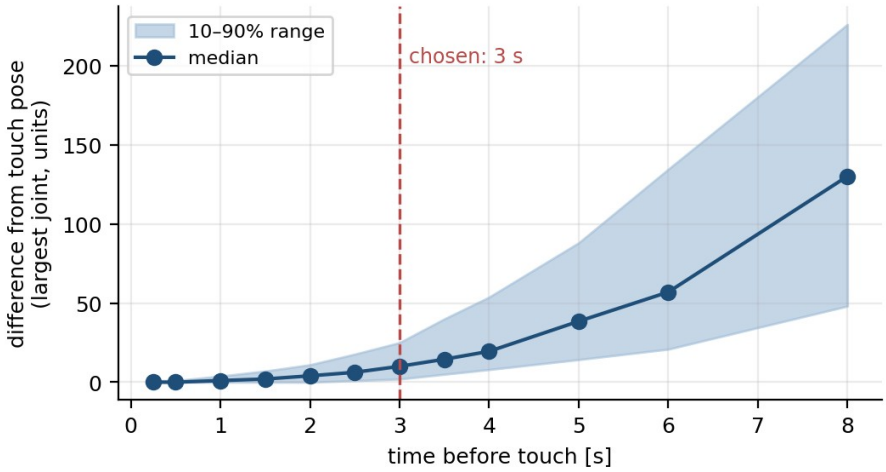


Figure 16. *Difference of the follower pose from the touch pose (largest joint) as a function of the time before the touch. The dashed line marks the chosen 3 s.*

9.2 Minimum-jerk trajectory

Each motion between two poses follows a **minimum-jerk** profile (Flash and Hogan, 1985). Velocity and acceleration are zero at the start and end of the motion and the total *jerk* (derivative of acceleration) energy is minimized:

$$\min \int_0^T \left(\frac{d^3q}{dt^3}\right)^2 dt \;\Rightarrow\; q(t) \text{ is a degree 5 polynomial} \tag{24}$$

Given the boundary conditions (position, velocity, acceleration), the solution of this variational problem is a degree-5 polynomial; with normalized time $\tau = t/T$:

$$s(\tau) = 10\tau^3 - 15\tau^4 + 6\tau^5, \qquad \tau = t/T \tag{25}$$

$$\frac{ds}{d\tau} = 30\,\tau^2(1-\tau)^2, \qquad \max_t \dot{q} = \frac{15}{8}\frac{|\Delta q|}{T} \tag{26}$$

$$\mathbf{q}(t) = \mathbf{q}_0 + (\mathbf{q}_1 - \mathbf{q}_0)s(t/T) \tag{27}$$

The velocity profile is bell-shaped and its peak at τ = 0.5 is 1.875 times the mean velocity (Figure 17). This makes the motion on the real arm start and stop smoothly, i.e. it does not strain the servos and the mechanics.

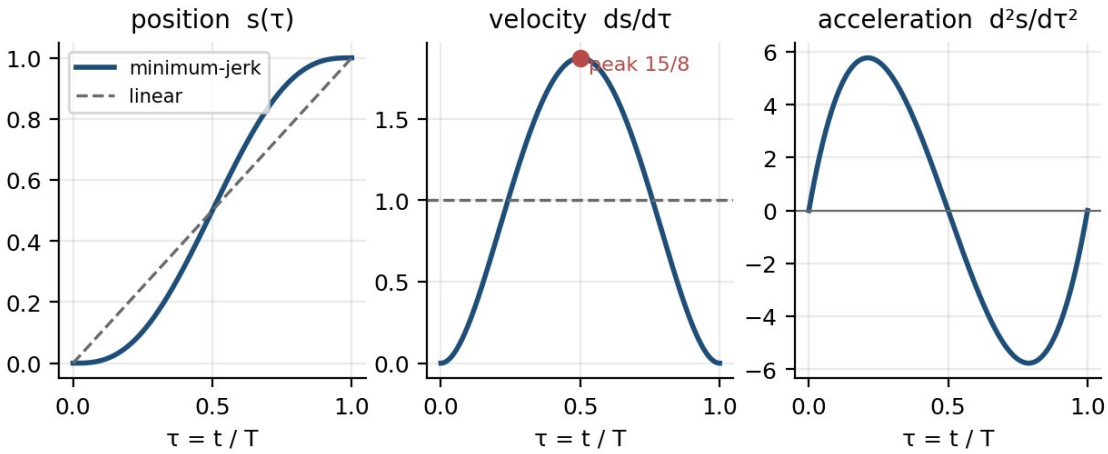


Figure 17. *Minimum-jerk profile: position, velocity and acceleration (dashed line: linear interpolation).*

Speed check: the largest joint motion is about 300 units for the accepted dots; in the 6 s approach phase the peak velocity is ≈ 1.875 · 300 / 6 ≈ 94 units/s (≈ 27°/s), well below the firmware's PC-mode limit (300 units/s).

9.3 Motion phases

Table 17. *Run phases (--speed 1)*

Phase	Duration	Start → end	Note
Approach	6.0 s	home → approach pose	all joints together (linear in joint space)
Touch	4.0 s	approach → touch pose	small motion (≈ 10 units), slow descent
Hold	0.7 s	at the touch pose	contact time
Lift	3.0 s	touch → approach	
Return	6.0 s	approach → home	the firmware also homes separately

The `--speed` parameter divides all durations (0.5 → twice as slow, ≈ 39 s in total). The total duration for *--speed 1* is 19.7 s.

9.4 Safety layers

Table 18. *Safety when running the model*

Layer	Measure
PC (run_model.py)	the dot is **rejected** if it is farther than 0.12 table units from the nearest training point; goals are clipped to [100, 923]; **x / ESC** emergency stop (the arm stays in place, homing separately with `h`)
Firmware	PC-mode slew limit (300 units/s); HOLD if PC commands stop for 300 ms; tracking error > 80 units for 500 ms → HOLD; voltage and temperature emergency stop
Human	cutting the power; first runs with `--speed 0.5` and a hand on the emergency stop key

10. Deployment on the real arm

10.1 Run flow

```
python3 tools/run_model.py 0 --speed 0.5      # camera + real arm

camera finds (u, v) → model predicts the touch pose → approach pose = touch + δ
→ [home → approach → touch → hold → lift → home]  (G lines at 40 Hz)
```

The camera finds the dot; if the median of the last 15 frames is stable, the model predicts the touch pose. The motion starts with SPACE or the button; every run is saved under `data/runs/` together with the goals sent and the measured follower positions.

10.2 Runs

There are 17 run records in total: **9 runs were completed** and **8 runs were aborted before moving**. Each abort had the same cause, “could not enter PC mode”: the arm did not switch to PC mode within 3 s. The firmware only accepts the `p` command in the WAIT, HOLD and READY states; if the arm is in another state (for example still homing) the command is ignored. The cause of these aborts was not investigated further in this report.

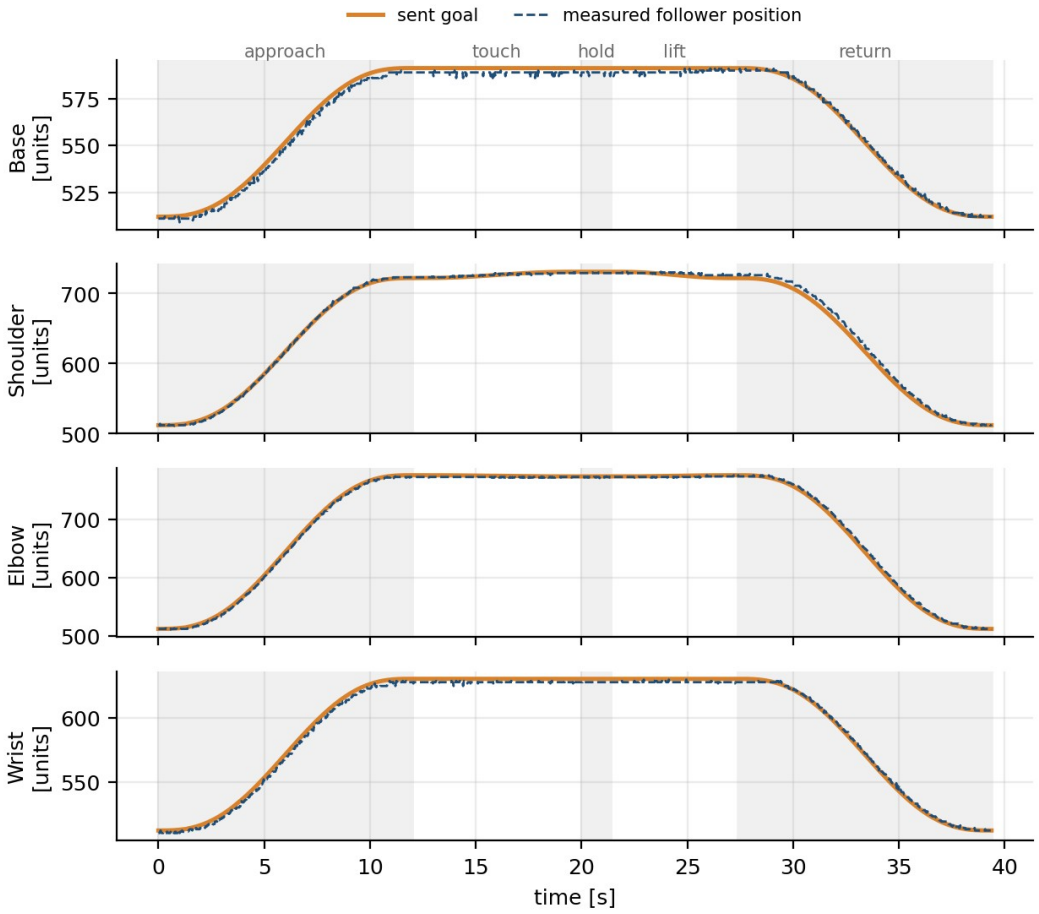


Figure 18. *A completed run on the real arm (run_1020_231600; dot u = 0.26, v = 0.30; --speed 0.5). Orange: goal sent; dashed: measured follower position. Shaded areas separate the phases.*

The motion is smooth, following the minimum-jerk profile, and the follower tracks the goal closely: in this run the median goal–position difference is 1.8 – 2.6 units depending on the joint, and the 95th percentile is at most 8.6 units (shoulder).

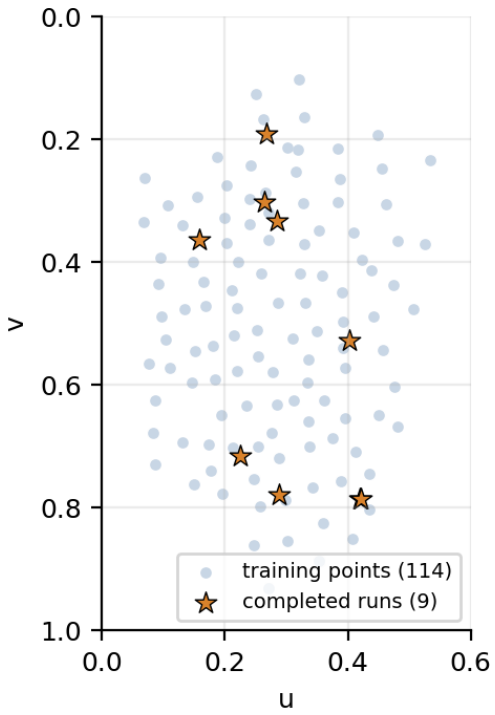


Figure 19. *Training points (light blue) and runs completed on the real arm (orange stars).*

The completed runs cover a large part of the reach area (approximately u = 0.16–0.42, v = 0.19–0.79).

10.3 Evaluation

According to the user, the arm found and touched the dot in all completed runs. This is consistent with the expected error of 1–1.5 cm from Section 8 (dot diameter ≈ 2.5–3 cm). **There is no quantitative measurement**: the touches were judged by eye and no end-point error in millimetres is given in this report.

11. Limitations and possible improvements

1. **Quantitative accuracy was not measured.** Success was judged by eye. Finding the screw tip in the camera image and computing its distance to the dot automatically would give the error in millimetres and allow a fair comparison with other methods.
2. **Risk of path collisions.** The trajectory uses linear interpolation in joint space; since the arm geometry is not modelled, it is theoretically possible for the tip to come close to the table mid-path. No problems have been seen so far at low speed. Next step: build a planar forward-kinematics model and plan the path in Cartesian space.
3. **Extrapolation is unreliable.** The model is only valid in the region seen in training ($u \approx 0.07$–0.53, $v \approx 0.10$–0.93); this region is also the reach area of the arm.
4. **Small data set (114 episodes).** The neural network lost to the polynomial with this data. More complex tasks will need more data or a different method.
5. **The wrist posture is not chosen.** The model produces the mean along the redundancy direction. This is valid as long as the curvature of the set is small; near the joint limits the mean may be an invalid pose. Fixing the wrist angle with a constraint (e.g. a fixed approach angle relative to the dot) and solving the other joints analytically would remove this.
6. **Dead zone.** A basic limitation of the AX-12A; a joint whose working range approaches 0 or 1023 must be fixed by remounting.
7. **Power.** A large part of the servo problems came from a drained battery. Collecting data with a stable 12 V power supply makes the data more consistent.
8. **An image-based policy was not tried.** The video was recorded but not used; since the dot position is already known, the image adds no information for this task. When tasks become multi-step and variable (e.g. pick and place), ACT/diffusion-style methods become meaningful.
9. **Aborted runs.** The cause of the 8 runs aborted with “could not enter PC mode” was not investigated; most likely the command was sent before the arm was ready.

12. Conclusion

In this work a two-arm leader–follower system was built, a pipeline that measures table coordinates from images was developed, 114 usable demonstrations were collected and a model was trained that predicts the touch pose from the dot position. Despite per-joint errors that look misleading because of the arm's redundancy, the model worked with an accuracy corresponding to an end-point error of about 3° (≈ 1–1.5 cm) and was deployed on the real arm.

The most instructive findings of the project are: (i) for a single-step, static task a simple regression beats a neural network; (ii) per-joint error measures are misleading in the presence of redundancy, and an eigen-analysis of the error covariance reveals the actual error; (iii) the power supply directly determines data quality; (iv) the same infrastructure (home pose, data stream, button, monitoring, coverage map, PC mode) can be reused directly for the next projects — target tracking, pick and place, drawing.

Appendix A. Symbols

Table 19. *Symbols used in the report*

Symbol	Meaning
p	servo position (0–1023)
l_i, f_i, g_i	leader position, follower position, goal sent to the follower (joint i)
H^L_i, H^F_i	leader and follower home value
s_i	joint direction sign (+1 / −1)
m_i, M_i	follower safe lower and upper limit
b_i, ℓ_i	goal before latency compensation, lead term
α, T_ℓ, ℓ_{max}	velocity filter coefficient, lead time, lead limit
R, Δt	slew-rate limit [units/s], loop period
$\mathbf{H}$	homography matrix (3×3)
(x, y), (u, v)	pixel coordinate, table coordinate (0–1)
A, P, C	component area, perimeter, circularity
$\mathbf{q}$	joint angle vector (base, shoulder, elbow, wrist)
$\mathbf{X}$, $\mathbf{Y}$, $\mathbf{W}$, φ, λ	feature matrix, target matrix, weights, feature vector, ridge coefficient
$\mathbf{C}$, $\mathbf{v}_k$, λ_k	error covariance, eigenvector, eigenvalue
τ, T, $s(\tau)$	normalized time, phase duration, minimum-jerk profile
$\boldsymbol{\delta}$	approach pose offset

Appendix B. File map and commands

Table 20. *Code structure*

File	Purpose
`firmware/arm_rec/arm_rec.ino`	**Main firmware**: home, teleop (leader–follower), data stream, button, PC mode, safety
`firmware/servo_lab/servo_lab.ino`	servo inspection console (`scan, info, watch, pair, go, sweep, home, follow, comm, range`)
`robot/opencm.py`	PC-side serial link; reads data lines, sends commands/goals
`vision/common.py`	ArUco wrapper (OpenCV 4.6 and 4.7+)
`vision/table_dot.py`	homography (`TableTracker`) and white dot detection (`find_dot`)
`vision/aruco_make.py`	generates the corner markers
`tools/record.py`	**data collection program**
`tools/run_model.py`	**runs the model**: find dot → predict pose → trajectory → arm
`tools/joint_range.py`	joint range and dead-zone check
`train/prepare.py`	cleans the data, extracts the touch/approach poses
`train/baseline.py`	compares the models with LOO
`report/make_figures.py`, `build_report.py`	generate the figures and the document of this report
`data/episodes/`, `data/processed/`, `data/runs/`	raw episodes, processed data set and plots, run records

Commands (from an Ubuntu terminal; the VS Code snap terminal can cause problems with the OpenCV window):

```
# Data collection
cd ~/Desktop/arm && python3 tools/record.py 0

# Prepare the data and compare the models
.venv/bin/python train/prepare.py
.venv/bin/python train/baseline.py

# Run the model
.venv/bin/python tools/run_model.py --uv 0.3 0.5 --dry --speed 0.5   # plan only
.venv/bin/python tools/run_model.py 0 --speed 0.5                     # real arm

# Rebuild this report
bash report/build_all.sh
```

Building the firmware: select *OpenCM9.04 Board* in the Arduino IDE; from the command line: `arduino-cli compile --fqbn OpenCM904:OpenCM904:OpenCM904 firmware/arm_rec`.

References

ROBOTIS. *Dynamixel AX-12A e-Manual* (Protocol 1.0, control table).

Garrido-Jurado, S., Muñoz-Salinas, R., Madrid-Cuevas, F. J., Marín-Jiménez, M. J. (2014). Automatic generation and detection of highly reliable fiducial markers under occlusion. *Pattern Recognition*, 47(6).

Hartley, R., Zisserman, A. (2003). *Multiple View Geometry in Computer Vision*, 2nd edition. Cambridge University Press. (Homography, DLT.)

Hoerl, A. E., Kennard, R. W. (1970). Ridge regression: biased estimation for nonorthogonal problems. *Technometrics*, 12(1).

Rasmussen, C. E., Williams, C. K. I. (2006). *Gaussian Processes for Machine Learning*. MIT Press.

Flash, T., Hogan, N. (1985). The coordination of arm movements: an experimentally confirmed mathematical model. *Journal of Neuroscience*, 5(7).

Zhao, T. Z., Kumar, V., Levine, S., Finn, C. (2023). Learning fine-grained bimanual manipulation with low-cost hardware (ACT). *RSS*.

OpenCV documentation: `getPerspectiveTransform`, `aruco`, `moments`.